

Digital Nurture 5.0 - Java FSE React (Week-1 Work)

Data Structures and Algorithm:

Exercise 1: Inventory Management System

Scenario:

You are developing an inventory management system for a warehouse. Efficient data storage and retrieval are crucial.

Steps:

1. Understand the Problem:

- Explain why data structures and algorithms are essential in handling large inventories.
- Discuss the types of data structures suitable for this problem.

2. Setup:

- Create a new project for the inventory management system.

3. Implementation:

- Define a class Product with attributes like **productId**, **productName**, **quantity**, and **price**.
- Choose an appropriate data structure to store the products (e.g., ArrayList, HashMap).
- Implement methods to add, update, and delete products from the inventory.

4. Analysis:

- Analyze the time complexity of each operation (add, update, delete) in your chosen data structure.
- Discuss how you can optimize these operations.

Solution:

CODE:

```
import java.util.HashMap;
```

```
import java.util.Map;
```

```
class Product {
```

```
int productId;
```

```
String productName;
```

```
int quantity;
```

```
double price;
```

```
public Product(int productId, String productName,  
               int quantity, double price) {
```

```
    this.productId = productId;
```

```
    this.productName = productName;
```

```
    this.quantity = quantity;
```

```
    this.price = price;
```

```
}
```

```
@Override
```

```
public String toString() {
```

```
    return "Product ID: " + productId +
```

```
        ", Product Name: " + productName +
```

```
        ", Quantity: " + quantity +
```

```
        ", Price: " + price;
```

```
}
```

```
}
```

```
class Inventory {
```

```
    private HashMap<Integer, Product> products = new HashMap<>();
```

```
    public void addProduct(Product product) {
```

```
        products.put(product.productId, product);
```

```
        System.out.println("Product Added.");
```

```
}
```

```
public void updateProduct(int id, int quantity, double price) {
```

```
    Product product = products.get(id);
```

```
    if (product != null) {
```

```
        product.quantity = quantity;
```

```
        product.price = price;
```

```
        System.out.println("Product Updated.");
```

```
    } else {
```

```
        System.out.println("Product Not Found.");
```

```
    }
```

```
}
```

```
public void deleteProduct(int id) {
```

```
    if (products.remove(id) != null) {
```

```
        System.out.println("Product Deleted.");
```

```
    } else {
```

```
        System.out.println("Product Not Found.");
```

```
    }
```

```
}
```

```
public void displayInventory() {
```

```
    System.out.println("\nInventory Details:");
```

```
    for (Map.Entry<Integer, Product> entry : products.entrySet()) {
```

```
        System.out.println(entry.getValue());
```

```
    }
```

```
}
```

```
}
```

```
public class InventoryManagementSystem {
```

```
    public static void main(String[] args) {
```

```
        Inventory inventory = new Inventory();
```

```
        Product p1 = new Product(
```

```
            101,
```

```
            "Laptop",
```

```
            10,
```

```
            50000.0);
```

```
        Product p2 = new Product(
```

```
            102,
```

```
            "Mouse",
```

```
            50,
```

```
            500.0);
```

```
        inventory.addProduct(p1);
```

```
        inventory.addProduct(p2);
```

```
        inventory.displayInventory();
```

```
        inventory.updateProduct(
```

```
            101,
```

```
            15,
```

```
            52000.0);
```

```
        inventory.displayInventory();
```

```

        inventory.deleteProduct(102);

        inventory.displayInventory();
    }
}

```

OUTPUT:

```

PS C:\Users\shiva\OneDrive\Desktop\Data Structures and Algorithm\Codes> javac InventoryManagementSystem.java
PS C:\Users\shiva\OneDrive\Desktop\Data Structures and Algorithm\Codes> java InventoryManagementSystem
Product Added.
Product Added.

Inventory Details:
Product ID: 101, Product Name: Laptop, Quantity: 10, Price: 50000.0
Product ID: 102, Product Name: Mouse, Quantity: 50, Price: 500.0
Product Updated.

Inventory Details:
Product ID: 101, Product Name: Laptop, Quantity: 15, Price: 52000.0
Product ID: 102, Product Name: Mouse, Quantity: 50, Price: 500.0
Product Deleted.

Inventory Details:
Product ID: 101, Product Name: Laptop, Quantity: 15, Price: 52000.0
PS C:\Users\shiva\OneDrive\Desktop\Data Structures and Algorithm\Codes> 

```

Exercise 2: E-commerce Platform Search Function

Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Steps:

1. **Understand Asymptotic Notation:**
 - Explain Big O notation and how it helps in analyzing algorithms.
 - Describe the best, average, and worst-case scenarios for search operations.
2. **Setup:**
 - Create a class **Product** with attributes for searching, such as **productId**, **productName**, and **category**.
3. **Implementation:**
 - Implement linear search and binary search algorithms.
 - Store products in an array for linear search and a sorted array for binary search.

4. Analysis:

- Compare the time complexity of linear and binary search algorithms.
- Discuss which algorithm is more suitable for your platform and why.

Solution:

CODE:

```
class Product {

    int productId;

    String productName;

    String category;

    public Product(int productId,
                   String productName,
                   String category) {

        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }

    @Override
    public String toString() {
        return "Product ID: " + productId +
            ", Name: " + productName +
            ", Category: " + category;
    }
}

public class ECommerceSearch {

    public static Product linearSearch(Product[] products,
```

```
        int targetId) {

    for (Product product : products) {

        if (product.productId == targetId) {
            return product;
        }
    }

    return null;
}

public static Product binarySearch(Product[] products,
        int targetId) {

    int low = 0;
    int high = products.length - 1;

    while (low <= high) {

        int mid = (low + high) / 2;

        if (products[mid].productId == targetId) {
            return products[mid];
        }

        if (products[mid].productId < targetId) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
}
```

```
}
```

```
return null;
```

```
}
```

```
public static void main(String[] args) {
```

```
    Product[] products = {
```

```
        new Product(101,
```

```
            "Laptop",
```

```
            "Electronics"),
```

```
        new Product(102,
```

```
            "Mouse",
```

```
            "Electronics"),
```

```
        new Product(103,
```

```
            "Keyboard",
```

```
            "Electronics"),
```

```
        new Product(104,
```

```
            "Shoes",
```

```
            "Fashion"),
```

```
        new Product(105,
```

```
            "Watch",
```

```
            "Accessories")
```

```
    };
```

```
    int targetId = 104;
```



```
System.out.println("Linear Search:");
```

```
Product result1 =
```

```
    linearSearch(products,  
        targetId);
```

```
if (result1 != null) {
```

```
    System.out.println(result1);
```

```
} else {
```

```
    System.out.println("Product Not Found");
```

```
}
```

```
System.out.println();
```

```
System.out.println("Binary Search:");
```

```
Product result2 =
```

```
    binarySearch(products,  
        targetId);
```

```
if (result2 != null) {
```

```
    System.out.println(result2);
```

```
} else {
```

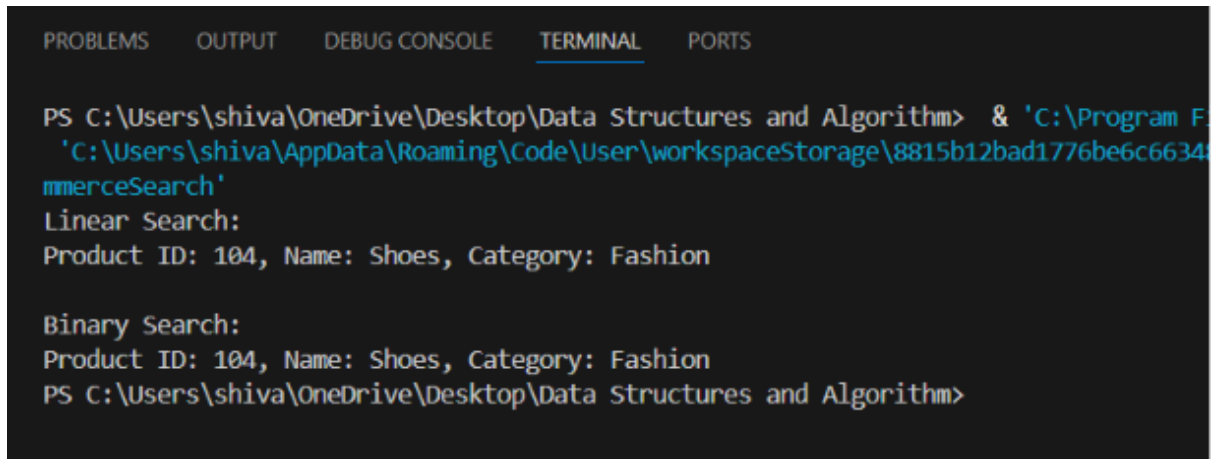
```
    System.out.println("Product Not Found");
```

```
}
```

```
}
```

```
}
```

OUTPUT:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\shiva\OneDrive\Desktop\Data Structures and Algorithm> & 'C:\Program F
'C:\Users\shiva\AppData\Roaming\Code\User\workspaceStorage\8815b12bad1776be6c6634
mmerceSearch'
Linear Search:
Product ID: 104, Name: Shoes, Category: Fashion

Binary Search:
Product ID: 104, Name: Shoes, Category: Fashion
PS C:\Users\shiva\OneDrive\Desktop\Data Structures and Algorithm>
```

Exercise 3: Sorting Customer Orders

Scenario:

You are tasked with sorting customer orders by their total price on an e-commerce platform. This helps in prioritizing high-value orders.

Steps:

1. Understand Sorting Algorithms:

- Explain different sorting algorithms (Bubble Sort, Insertion Sort, Quick Sort, Merge Sort).

2. Setup:

- Create a class **Order** with attributes like **orderId**, **customerName**, and **totalPrice**.

3. Implementation:

- Implement **Bubble Sort** to sort orders by **totalPrice**.
- Implement **Quick Sort** to sort orders by **totalPrice**.

4. Analysis:

- Compare the performance (time complexity) of Bubble Sort and Quick Sort.
- Discuss why Quick Sort is generally preferred over Bubble Sort.

Solution:

CODE:

```
class Order {

    int orderId;

    String customerName;

    double totalPrice;

    public Order(int orderId,
                 String customerName,
                 double totalPrice) {

        this.orderId = orderId;

        this.customerName = customerName;

        this.totalPrice = totalPrice;

    }

    @Override
    public String toString() {
        return "Order ID: " + orderId +
            ", Customer: " + customerName +
            ", Total Price: " + totalPrice;
    }
}

public class OrderSortingSystem {

    public static void displayOrders(Order[] orders) {

        for (Order order : orders) {
            System.out.println(order);
        }
    }
}
```

```
    }  
}
```

```
public static void bubbleSort(Order[] orders) {
```

```
    int n = orders.length;
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        for (int j = 0; j < n - i - 1; j++) {
```

```
            if (orders[j].totalPrice >  
                orders[j + 1].totalPrice) {
```

```
                Order temp = orders[j];  
                orders[j] = orders[j + 1];  
                orders[j + 1] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
// Quick Sort
```

```
public static void quickSort(Order[] orders,
```

```
    int low,
```

```
    int high) {
```

```
    if (low < high) {
```

```
        int pivotIndex =
```

```
            partition(orders, low, high);
```

```

        quickSort(
            orders,
            low,
            pivotIndex - 1);

        quickSort(
            orders,
            pivotIndex + 1,
            high);
    }
}

public static int partition(Order[] orders,
                            int low,
                            int high) {

    double pivot =
        orders[high].totalPrice;

    int i = low - 1;

    for (int j = low; j < high; j++) {

        if (orders[j].totalPrice < pivot) {

            i++;

            Order temp = orders[i];
            orders[i] = orders[j];
            orders[j] = temp;

```

```
    }  
}
```

```
    Order temp = orders[i + 1];  
    orders[i + 1] = orders[high];  
    orders[high] = temp;  
  
    return i + 1;  
}
```

```
public static void main(String[] args) {
```

```
    Order[] orders1 = {
```

```
        new Order(  
            101,  
            "Rahul",  
            5000),
```

```
        new Order(  
            102,  
            "Anjali",  
            2500),
```

```
        new Order(  
            103,  
            "Kiran",  
            8000),
```

```
        new Order(  
            104,
```

```
        "Priya",  
        4000)  
};
```

```
System.out.println(  
    "Original Orders:";
```

```
displayOrders(orders1);
```

```
bubbleSort(orders1);
```

```
System.out.println(  
    "\nOrders After Bubble Sort:");
```

```
displayOrders(orders1);
```

```
Order[] orders2 = {
```

```
    new Order(  
        101,  
        "Rahul",  
        5000),
```

```
    new Order(  
        102,  
        "Anjali",  
        2500),
```

```
    new Order(  
        103,  
        "Kiran",
```

```

        8000),

        new Order(

            104,

            "Priya",

            4000)

    };

    quickSort(

        orders2,

        0,

        orders2.length - 1);

    System.out.println(

        "\nOrders After Quick Sort:");

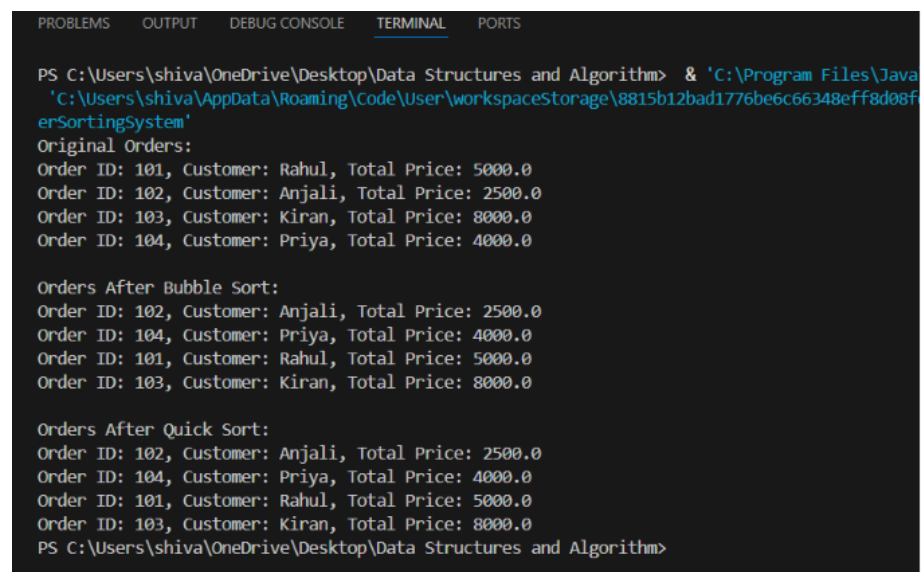
    displayOrders(orders2);

}

}

```

OUTPUT:



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\shiva\OneDrive\Desktop\Data Structures and Algorithm> & 'C:\Program Files\Java
'C:\Users\shiva\AppData\Roaming\Code\User\workspaceStorage\8815b12bad1776be6c66348eff8d08f
erSortingSystem'
Original Orders:
Order ID: 101, Customer: Rahul, Total Price: 5000.0
Order ID: 102, Customer: Anjali, Total Price: 2500.0
Order ID: 103, Customer: Kiran, Total Price: 8000.0
Order ID: 104, Customer: Priya, Total Price: 4000.0

Orders After Bubble Sort:
Order ID: 102, Customer: Anjali, Total Price: 2500.0
Order ID: 104, Customer: Priya, Total Price: 4000.0
Order ID: 101, Customer: Rahul, Total Price: 5000.0
Order ID: 103, Customer: Kiran, Total Price: 8000.0

Orders After Quick Sort:
Order ID: 102, Customer: Anjali, Total Price: 2500.0
Order ID: 104, Customer: Priya, Total Price: 4000.0
Order ID: 101, Customer: Rahul, Total Price: 5000.0
Order ID: 103, Customer: Kiran, Total Price: 8000.0
PS C:\Users\shiva\OneDrive\Desktop\Data Structures and Algorithm>

```